

ECOM EXPERTS

Ecom traceerrors

Manual de usuario, tecnico y de desarrollo

ecom_traceerrors · 08/09/2026

Contenido

1. Visión General

2. Arquitectura del Sistema

2.1. Configuración y UI (HelperForm + Tabs)

2.2. TraceLogger (Monitorización y Profiling)

2.3. Herramientas Integradas

2.4. AI Analyzer (Cerebro Adaptativo)

3. Estructura de Ficheros

Core

Vistas (Templates)

4. Notas de Seguridad

Introducción

1. Panel de Configuración

A. Configuration (Ajustes Generales)

B. Object Monitors (Monitores de Objetos)

2. Herramientas de Diagnóstico

Pestaña "Logs"

Pestaña "Scanner" (Buscador de Código)

Pestaña "System Profiling"

Pestaña "Auditor & Optimizer"

3. Preguntas Frecuentes

1. Estructura de Ficheros y Directorios

2. Flujo de Datos (Data Flow)

2.1. Captura de Eventos (Hooks)

2.2. Sistema "Giga Log" de Profiling

2.3. Motor de Comparación IA

2.4. Escritura en Disco (TraceLogger)

3. Seguridad y Privacidad

3.1. Protección de Logs

3.2. Sanitización de Datos

4. Limitaciones Conocidas

5. Sistema de Exportación PDF (v2.5+)

5.1. Clase ProfilingReportPDF

5.2. Métricas Analizadas

5.3. Endpoints AJAX Nuevos

5.4. Directorio de Reportes

6. Control Manual de Profiling (v2.5+)

6.1. Botón STOP

6.2. UI Mejorada

Arquitectura del Módulo

1. Sistema de Hooks (Hooks System)

2. TraceLogger (El Motor de Logs)

3. Extensión del Módulo

4. Depuración (Debug Mode)

Core Principles

Naming Rules

Function Rules

Code Structure

AI Coding Style

Anti-Patterns (DON'T)

● Before Editing ANY File (THINK FIRST!)

Summary

● Self-Check Before Completing (MANDATORY)

Verification Scripts (MANDATORY)

Agent → Script Mapping

● Script Output Handling (READ → SUMMARIZE → ASK)

Overview

When to Use This Skill

How It Works

Step 1: Understand the Context

Step 2: Review Functionality

Step 3: Review Code Quality

Step 4: Review Security

Step 5: Review Performance

Step 6: Review Tests

Examples

Example 1: Functionality Review Checklist

Example 2: Security Review Checklist

Example 3: Code Quality Review Checklist

Best Practices

✓ Do This

✗ Don't Do This

Complete Review Checklist

Pre-Review

Functionality

Security

Performance

Code Quality

Tests

Documentation

Git

Common Pitfalls

Problem: Missing Edge Cases

Problem: Security Vulnerabilities

Problem: Poor Test Coverage

Problem: Unclear Code

Review Comment Templates

Requesting Changes

Asking Questions

Praising Good Code

Related Skills

Additional Resources

1. Estándares de Codificación

2. Análisis Estático (PHPStan Level 9)

3. Reglas Específicas de Módulos

1. Visión General

Este módulo es una suite de depuración avanzada, auditoría de código y **monitorización de rendimiento** potenciada por IA para PrestaShop 1.7/8. A diferencia de los logs estándar, captura el *contexto operativo* (Trace, Actor, Source) e introduce un sistema de "**Inteligencia Adaptativa**" que selecciona automáticamente las habilidades (Skills) necesarias para analizar un error o cuello de botella.

2. Arquitectura del Sistema

2.1. Configuración y UI (HelperForm + Tabs)

El módulo ha evolucionado hacia una interfaz híbrida que combina la robustez nativa de `HelperForm` con la usabilidad de pestañas AJAX/Bootstrap: - **Configuration & Monitors**: Implementados 100% con `HelperForm` de PrestaShop para garantizar la integridad de los datos al guardar. Separados en dos pestañas lógicas ("Configuration" y "Object Monitors") para claridad visual. - **Herramientas (Tools)**: Logs, Scanner, Profiling y Auditor residen en plantillas `.tpl` independientes cargadas dinámicamente, manteniendo el código limpio y modular.

2.2. TraceLogger (Monitorización y Profiling)

La clase `TraceLogger` cubre áreas críticas con **Monitores Granulares** configurables:

A. Logs de Traza (`PrestaShopLogger.log`)

- **Monitores de Objetos (Object Monitors)**:
 - Interruptores independientes para activar/desactivar logs de:
 - **Productos, Combinaciones, Características**.
 - **Stock** (StockAvailable, Warehouse).
 - **Imágenes**: Lógica integrada en `processLog` para capturar detalles enriquecidos (ID, Posición, Cover) y respetar estrictamente la configuración.
 - **Precios**: Detecta cambios en Precio Base y Precios Específicos.
 - **Import**: Monitorización de payloads de importación masiva.
- **Exclusión Inteligente**: Permite ignorar clases u objetos específicos para reducir ruido.
- **Log Rotation**: Rota automáticamente logs > 5MB.

B. Profiling System (`logs/_profiling_YYYY-MM-DD.log`)

- **Perfilado de 10 Minutos**: Herramienta activable bajo demanda que registra el rendimiento de cada petición.
- **Métricas**: Tiempo de ejecución (microsegundos), Pico de Memoria, URI y User Agent.
- **Clasificación**: Detecta usuarios vs Bots (Googlebot, Bingbot, AI Bots) y marca peticiones LENTAS (>0.8s) o CRÍTICAS (>2.0s).

2.3. Herramientas Integradas

Code Scanner (Análisis Estático)

- Escaneo local de directorios (Clases, Módulos, Controllers) usando **Heurística Regex**.
- Detecta vulnerabilidades comunes (SQL Injection, XSS, `p()` / `d()` leftovers) sin coste de API.
- Opción de escalar hallazgos a la IA para verificación.

AI Module Auditor

- Sube un ZIP de módulo para auditoría instantánea.
- Modos: General (Clean Code), Performance (Optimización), Security (Vulnerabilidades).
- Análisis contextual usando Skills externas.

2.4. AI Analyzer (Cerebro Adaptativo)

El motor de IA (`classes/AIAnalyzer.php`) es **Context-Aware**: - **Smart Truncation**: Gestiona logs grandes mediante lectura de Cabecera/Cola. - **Intelligent Skill Selection**: Carga skills específicas (`Security_Review` , `Performance_Profiling`) basadas en el tipo de análisis solicitado. - **Integración Dual**: Soporta tanto Google Gemini como OpenAI (GPT-4).

3. Estructura de Ficheros

Core

- `ecom_traceerrors.php` : Controlador principal, gestiona Hooks, AJAX handlers y generación de Forms (`renderSettingsForm` , `renderMonitorsForm`).
- `classes/TraceLogger.php` : Motor de logs y profiling.
- `classes/AIAnalyzer.php` : Cliente API y gestor de contexto.

Vistas (Templates)

- `views/templates/admin/configure.tpl` : Layout maestro con navegación por pestañas y lógica de visualización de errores.
 - `views/templates/admin/tab_logs.tpl` : Visor y gestor de ficheros de log.
 - `views/templates/admin/tab_scanner.tpl` : Interfaz del Escáner de Código.
 - `views/templates/admin/tab_profiling.tpl` : Interfaz del Profiler de Sistema.
 - `views/templates/admin/tab_auditor.tpl` : Interfaz de Auditoría de Módulos (Upload ZIP).
 - `views/templates/admin/modal.tpl` : Lógica JavaScript compartida y modales de respuesta AI.
-

4. Notas de Seguridad

- **Validación Estricta**: Todas las operaciones de archivo (descarga, borrado, escaneo) validan rutas absolutas y previenen Path Traversal.

- **Sanitización:** Los inputs de configuración se validan mediante `Tools::getValue` y cast explícito (`(bool)` , `(int)`).
 - **Acceso:** Restringido a empleados con permisos de configuración de módulos.
-

Autor: Gravity Architect **Versión Actual:** 1.0.7 (Granular Object Monitors & Static Tools Integration)

ECOM TRACE ERRORS - Guía de Usuario

Introducción

Ecom Trace Errors es una herramienta avanzada para tener control total sobre lo que ocurre en tu tienda PrestaShop. No solo muestra errores, sino que te dice **quién** hizo qué, **cuándo** y **dónde**.

1. Panel de Configuración

El módulo se divide en pestañas fáciles de usar:

A. Configuration (Ajustes Generales)

Aquí defines el comportamiento global de la IA y los logs. - **Log Level:** - **Errors Only** : Solo guarda fallos críticos. Ahorra espacio. - **All (Debug & Info)** : Guarda TODO. Úsalo si estás investigando un problema. - **All (Debug & Info)** : Guarda TODO. Úsalo si estás investigando un problema. - **Test IP Address (Sandbox Mode)**: - Introduce TU dirección IP aquí. - **Efecto Mágico**: Si pones tu IP, el módulo ignorará a todos los demás usuarios y bots. Solo registrará TUS acciones. - Ideal para hacer pruebas de rendimiento limpias sin "ruido" de tráfico real. - **AI Settings**: Configura tu proveedor (Gemini/OpenAI) para analizar los errores.

B. Object Monitors (Monitores de Objetos)

Activa o desactiva la vigilancia sobre elementos específicos de tu tienda. - **Productos**: Cambios en descripción, SEO, categorías. - **Stock**: Movimientos de inventario. - **Imágenes**: - ¡Nuevo! Registra subidas, borrados y cambios de posición. - Guarda detalles como ID del producto asociado y si es portada. - Archivo de log: **ImageMonitor.log**. - **Precios**: Detecta cambios en el precio base y reglas de precios específicos. - **Import**: Monitoriza importaciones masivas (CSV/ERP).

2. Herramientas de Diagnóstico

Pestaña "Logs"

El visor central de eventos. - **Lista de Archivos**: Verás logs separados por tipo (**Product.log** , **ImageMonitor.log** , **Error.log**). - **Botones**: - **Ver** : Abre el contenido en pantalla. - **Descargar** : Baja el archivo **.log** a tu PC. - **Borrar** : Elimina logs antiguos. - **Analizar con IA** : Envía el fragmento a la IA para que te explique qué pasó.

Pestaña "Scanner" (Buscador de Código)

Para programadores o usuarios avanzados. Busca texto dentro de los archivos de tu tienda. - Útil para encontrar "dónde se define esta función" o "qué archivo contiene este error".

Pestaña "System Profiling"

Mide la velocidad de tu tienda. - Actívalo por 10 minutos si notas lentitud. - Generará un informe (`_profiling_x.log`) marcando en rojo las páginas lentas. - **¡NUEVO! Modo Comparación:** 1. Haz un profiling ANTES de optimizar (Base Line). 2. Haz cambios en tu tienda (activa caché, desactiva módulo lento, etc.). 3. Haz un profiling DESPUÉS. 4. Selecciona ambos logs en la lista y pulsa **"Compare Selected Logs"**. 5. La IA leerá ambos y te dará un informe detallando exactamente cuánto has mejorado (o empeorado).

Pestaña "Auditor & Optimizer"

- Sube un módulo (ZIP) para que la IA lo revise antes de instalarlo.
 - Detecta fallos de seguridad o código basura.
-

3. Preguntas Frecuentes

P: ¿Por qué no veo logs de imágenes? R: Asegúrate de que el interruptor "Monitor Images" está activo en la pestaña "Object Monitors". Si acabas de actualizar, recarga la página de configuración para que el módulo se "auto-repare" y registre los eventos necesarios.

P: ¿Los logs ocupan mucho espacio? R: El módulo tiene un sistema de rotación automática. Si un archivo supera los 5MB, lo archiva (.1.log) y crea uno nuevo, manteniendo tu servidor limpio.

P: ¿Es seguro tener los logs ahí? R: Sí. La carpeta `logs` está protegida para que nadie pueda verlos desde el navegador (Error 403 Forbidden). Solo tú puedes verlos desde el admin.

ECOM TRACE ERRORS - Especificación Técnica V2

1. Estructura de Ficheros y Directorios

```
ecom_traceerrors/  
├── classes/  
│   ├── AIAnalyzer.php      # Cliente API (OpenAI/Gemini) & Skill Manager (Dynamic Loader)  
│   ├── TraceLogger.php     # Motor de Logging, Profiling (GigaLog), Rotación y Seguridad  
│   └── CodeScanner.php      # Explorador de código fuente local  
├── docs/  
│   └── # Documentación (User, Dev, Tech)  
├── logs/  
│   ├── .htaccess           # Deny from all  
│   ├── Product.log  
│   ├── ImageMonitor.log  
│   └── ...  
├── skills/                  # Base de Conocimiento MD para la IA  
├── views/  
│   └── templates/  
│       ├── admin/  
│       │   ├── configure.tpl # Layout Principal (Tabs Container)  
│       │   ├── tab_logs.tpl  
│       │   └── ...  
├── ecom_traceerrors.php     # Controlador Principal  
└── TECHNICAL_MANUAL.md      # Visión General
```

2. Flujo de Datos (Data Flow)

2.1. Captura de Eventos (Hooks)

- Trigger:** PrestaShop dispara un hook (ej. `actionObjectAddAfter`).
- Controller:** `ecom_traceerrors.php` lo recibe. - Si es específico (ej. Imagen), va a su método dedicado (`monitorImage`). - Si es genérico, va a `processLog`.
- Validación:** - ¿Está el monitor activo en `Configuration`? - ¿Está el objeto en la lista de exclusión?
- Logging:** Llamada a `TraceLogger::log($type, $id, $info)`.
- Clean Code IP Filter:** Validación centralizada `shouldLog()` que permite filtrar TODA actividad si `ECOM_TRACE_TEST_IP` está configurada y no coincide (Modo Sandbox).

2.2. Sistema "Giga Log" de Profiling

- Dispatcher Intercep:** `hookActionDispatcher` captura inicio de ejecución.
- Forensic Trace:** Captura inmediata de 2 pasos de `debug_backtrace` para identificar overrides o puntos de entrada no estándar.
- Shutdown Metrics:** Al finalizar el script, registra: - Tiempo de ejecución y Memoria (Peak & Diff). - Conteo de archivos PHP cargados y Queries (si `Db` lo permite). - Metadatos expandidos: IP, UserAgent, Referer, Bot Type.

4. **HTML output:** Generación de informes enriquecidos con CSS (Courier New, 18px) para legibilidad directa.

2.3. Motor de Comparación IA

1. **Frontend:** Selección múltiple de logs (Before/After) via checkbox.
2. **Backend:** `ajaxCompareLogs` agrupa el contenido de los logs y añade una instrucción de comparación.
3. **IA Context:** Usa todos los skills disponibles (`skills/*.md`) para evaluar mejoras en memoria y tiempo.
4. **Resultado:** Genera un informe diferencial HTML.

2.4. Escritura en Disco (TraceLogger)

1. **Contexto:** `TraceLogger` captura el Stack Trace, Actor (Empleado/API) y Origen (Módulo/Core).
2. **Resolución de Nombre:** Aplica la lista de prioridad (`$priorityMonitors`) para decidir el nombre del archivo.
3. **Escritura:** - Verifica existencia de directorio. - Aplica rotación si tamaño > 5MB. - Escribe con `FILE_APPEND` | `LOCK_EX` .

3. Seguridad y Privacidad

3.1. Protección de Logs

- Implementada mediante `.htaccess` híbrido (Apache 2.2/2.4) en `/logs/` .
- Previene acceso directo a información sensible (emails, montos) que pudiera haber en los logs.
- PHP previene `Path Traversal` con `basename()` estricto al descargar logs.

3.2. Sanitización de Datos

- **Webservice:** Los payloads JSON/XML enviados a la API se capturan pero se **truncan** a 50KB y se **enmascaran** (claves 'passwd', 'key', 'token' se reemplazan por asteriscos) antes de guardar.

4. Limitaciones Conocidas

- **Hooks no estándar:** Algunos módulos de importación masiva escriben directamente en SQL (`Db::getInstance()->execute`), saltándose el `ObjectModel` y por tanto los hooks. Estos cambios no pueden ser traceados a menos que se usen Overrides de la clase `Db` (no implementado por estabilidad).
- **Rendimiento:** En nivel de log "Debug", el Profiling exhaustivo puede añadir ~5-10ms por petición debido al I/O de disco y Stack Tracing. Se recomienda usar nivel "Errors Only" en producción.

5. Sistema de Exportación PDF (v2.5+)

5.1. Clase ProfilingReportPDF

Nueva clase ubicada en `classes/ProfilingReportPDF.php` que implementa:

- **Parseo de logs** de profiling con extracción de métricas
- **Análisis de Core Web Vitals** (LCP, Memory) según estándares Google
- **Generación de PDF** usando TCPDF (nativo PrestaShop)
- **Fallback a HTML** si TCPDF no está disponible
- **Recomendaciones automáticas** basadas en umbrales de rendimiento

5.2. Métricas Analizadas

Métrica	Target	Fuente
LCP (Response Time)	< 2.5s	web-performance-optimization skill
Memory Peak	< 128 MB	backend-dev-guidelines skill
Slow Requests %	< 10%	performance-profiling skill
Bot vs User Traffic	Balance	analytics-tracking skill

5.3. Endpoints AJAX Nuevos

```
// Exportar PDF
action=export_pdf&filename=system_profile_XXXXX.log

// Reporte rápido JSON
action=performance_report&filename=system_profile_XXXXX.log
```

5.4. Directorio de Reportes

```
ecom_traceerrors/
├─ reports/          # Nuevo: Almacena PDFs/HTML generados
│  └─ index.php      # Seguridad
│  └─ system_profile_*_report.pdf
```

6. Control Manual de Profiling (v2.5+)

6.1. Botón STOP

Permite detener el profiling manualmente en cualquier momento:

```
action=stop_profiling // Establece ECOM_TRACE_PROFILING_STOP = time() - 1
```

6.2. UI Mejorada

- **Barra de progreso** visual con porcentaje
- **Timer en tiempo real** (formato MM:SS)

- **Contador de logs** recopilados
- **Hora de finalización** estimada

ECOM TRACE ERRORS - Guía de Desarrollo

Arquitectura del Módulo

Este módulo sigue una arquitectura híbrida entre los estándares clásicos de PrestaShop (1.6/1.7) y patrones modernos de monitoreo.

1. Sistema de Hooks (Hooks System)

El módulo implementa mecanismos de escucha duales:

A. Generic Hooks (`actionObject*`)

- `actionObjectAddAfter` , `actionObjectUpdateAfter` , `actionObjectDeleteAfter` .
- **Propósito:** Capturar cambios en cualquier entidad (`ObjectModel`) sin necesidad de registrar hooks específicos para cada una.
- **Flujo:** Todo pasa por el método central `processLog($params, $action)` .
- **Ventaja:** Cobertura total (Custom clases, módulos de terceros).

B. Specific Monitors (`monitor*`)

- **Imágenes:** Usa hooks específicos (`actionObjectImageAddAfter`) para garantizar la captura de eventos que a veces no activan el hook genérico debido a implementaciones no estándar en controladores o temas.
- **Precios Específicos:** `actionObjectSpecificPriceAddAfter` .
- **Lógica de "Auto-Repair":** En `getContent()` , el módulo verifica si estos hooks específicos están registrados. Si no (por ejemplo, tras una actualización manual de archivos), los registra silenciosamente "on-the-fly".

2. TraceLogger (El Motor de Logs)

Ubicación: `classes/TraceLogger.php` .

Prioridad de Monitores (`$priorityMonitors`)

Para evitar que los logs sean renombrados erróneamente (ej. `Module_ps_facetedsearch.log` en lugar de `Product.log` cuando el módulo de facetas actualiza un producto), `TraceLogger` implementa una **Lista Blanca de Prioridad**:

```
$priorityMonitors = array(
    'ImageMonitor', 'PriceMonitor', 'ImportMonitor',
    'Product', 'Stock', '...CoreObjects...'
);
```

Si el objeto logueado está en esta lista, el nombre del archivo se respeta **siempre**.

Detección de Actor y Origen

- `detectActor()` : Discrimina entre Empleado, Cliente, Webservice (API Auth) y CLI/Cron.

- `detectSource()` : Analiza el `debug_backtrace` para identificar si la llamada partió del Core, un Módulo (extrae el nombre) o un Override.

3. Extensión del Módulo

Añadir un Nuevo Monitor

1. **Configuración:** Añade el campo `switch` en `renderMonitorsForm()` (archivo `ecom_traceerrors.php`).
2. **Procesamiento:** - Opción A (Ligera): Añade un `if ($className === 'MiClase')` en `processLog` . - Opción B (Robusta): Crea hooks específicos `actionObjectMiClase...` y regístralos en `getContent` (Auto-Repair).
3. **TraceLogger:** Añade 'MiClase' a `$priorityMonitors` en `TraceLogger.php` si quieres un archivo dedicado exclusivo.

4. Depuración (Debug Mode)

El módulo soporta niveles de log: - `Configuration::get('ECOM_TRACE_LOG_LEVEL')` - `0` : Solo errores. - `1` : Traza completa.

En `processLog` , el nivel 1 activa `DEBUG_TRACE` para seguir el flujo de ejecución de hooks, útil para saber si un monitor está activo o siendo ignorado por exclusiones.

name: clean-code description: Pragmatic coding standards - concise, direct, no over-engineering, no unnecessary comments allowed-tools: Read, Write, Edit version: 2.0 priority: CRITICAL

Clean Code - Pragmatic AI Coding Standards

CRITICAL SKILL - Be **concise, direct, and solution-focused**.

Core Principles

Principle	Rule
SRP	Single Responsibility - each function/class does ONE thing
DRY	Don't Repeat Yourself - extract duplicates, reuse
KISS	Keep It Simple - simplest solution that works
YAGNI	You Aren't Gonna Need It - don't build unused features
Boy Scout	Leave code cleaner than you found it

Naming Rules

Element	Convention
Variables	Reveal intent: <code>userCount</code> not <code>n</code>
Functions	Verb + noun: <code>getUserById()</code> not <code>user()</code>
Booleans	Question form: <code>isActive</code> , <code>hasPermission</code> , <code>canEdit</code>
Constants	SCREAMING_SNAKE: <code>MAX_RETRY_COUNT</code>

Rule: If you need a comment to explain a name, rename it.

Function Rules

Rule	Description
Small	Max 20 lines, ideally 5-10
One Thing	Does one thing, does it well
One Level	One level of abstraction per function
Few Args	Max 3 arguments, prefer 0-2
No Side Effects	Don't mutate inputs unexpectedly

Code Structure

Pattern	Apply
Guard Clauses	Early returns for edge cases
Flat > Nested	Avoid deep nesting (max 2 levels)
Composition	Small functions composed together
Colocation	Keep related code close

AI Coding Style

Situation	Action
User asks for feature	Write it directly
User reports bug	Fix it, don't explain
No clear requirement	Ask, don't assume

Anti-Patterns (DON'T)

✗ Pattern	✓ Fix
Comment every line	Delete obvious comments
Helper for one-liner	Inline the code
Factory for 2 objects	Direct instantiation
utils.ts with 1 function	Put code where used
"First we import..."	Just write code
Deep nesting	Guard clauses
Magic numbers	Named constants
God functions	Split by responsibility

● Before Editing ANY File (THINK FIRST!)

Before changing a file, ask yourself:

Question	Why
What imports this file?	They might break
What does this file import?	Interface changes
What tests cover this?	Tests might fail
Is this a shared component?	Multiple places affected

Quick Check:

```
File to edit: UserService.ts
└─ Who imports this? → UserController.ts, AuthController.ts
└─ Do they need changes too? → Check function signatures
```

● **Rule:** Edit the file + all dependent files in the SAME task. ● **Never leave broken imports or missing updates.**

Summary

Do	Don't
Write code directly	Write tutorials
Let code self-document	Add obvious comments
Fix bugs immediately	Explain the fix first
Inline small things	Create unnecessary files
Name things clearly	Use abbreviations
Keep functions small	Write 100+ line functions

Remember: The user wants working code, not a programming lesson.

● Self-Check Before Completing (MANDATORY)

Before saying "task complete", verify:

Check	Question
✓ Goal met?	Did I do exactly what user asked?
✓ Files edited?	Did I modify all necessary files?
✓ Code works?	Did I test/verify the change?
✓ No errors?	Lint and TypeScript pass?
✓ Nothing forgotten?	Any edge cases missed?

● **Rule:** If ANY check fails, fix it before completing.

Verification Scripts (MANDATORY)

● **CRITICAL:** Each agent runs ONLY their own skill's scripts after completing work.

Agent → Script Mapping

Agent	Script	Command
frontend-specialist	UX Audit	python ~/.claude/skills/frontend-design/scripts/ux_audit.py .
frontend-specialist	A11y Check	python ~/.claude/skills/frontend-design/scripts/accessibility_checker.py .
backend-specialist	API Validator	python ~/.claude/skills/api-patterns/scripts/api_validator.py .
mobile-developer	Mobile Audit	python ~/.claude/skills/mobile-design/scripts/mobile_audit.py .
database-architect	Schema Validate	python ~/.claude/skills/database-design/scripts/schema_validator.py .
security-auditor	Security Scan	python ~/.claude/skills/vulnerability-scanner/scripts/security_scan.py .
seo-specialist	SEO Check	python ~/.claude/skills/seo-fundamentals/scripts/seo_checker.py .
seo-specialist	GEO Check	python ~/.claude/skills/geo-fundamentals/scripts/geo_checker.py .
performance-optimizer	Lighthouse	python ~/.claude/skills/performance-profiling/scripts/lighthouse_audit.py <url>
test-engineer	Test Runner	python ~/.claude/skills/testing-patterns/scripts/test_runner.py .
test-engineer	Playwright	python ~/.claude/skills/webapp-testing/scripts/playwright_runner.py <url>
Any agent	Lint Check	python ~/.claude/skills/lint-and-validate/scripts/lint_runner.py .
Any agent	Type Coverage	python ~/.claude/skills/lint-and-validate/scripts/type_coverage.py .
Any agent	i18n Check	python ~/.claude/skills/i18n-localization/scripts/i18n_checker.py .

 **WRONG:**

test-engineer running ux_audit.py

 **CORRECT:**

frontend-specialist running ux_audit.py

● Script Output Handling (READ → SUMMARIZE → ASK)

When running a validation script, you **MUST**:

1. **Run the script** and capture ALL output
2. **Parse the output** - identify errors, warnings, and passes
3. **Summarize to user** in this format:

```
## Script Results: [script_name.py]

### ❌ Errors Found (X items)
- [File:Line] Error description 1
- [File:Line] Error description 2

### ⚠️ Warnings (Y items)
- [File:Line] Warning description

### ✅ Passed (Z items)
- Check 1 passed
- Check 2 passed

**Should I fix the X errors?**
```

4. **Wait for user confirmation** before fixing
5. **After fixing** → Re-run script to confirm

● **VIOLATION:** Running script and ignoring output = FAILED task. ● **VIOLATION:** Auto-fixing without asking = Not allowed. ● **Rule:** Always READ output → SUMMARIZE → ASK → then fix.

IDENTITY

Eres **Refactor Doctor**, un experto en Clean Code y Arquitectura de Software.

CLEAN CODE PRINCIPLES

1. **DRY (Don't Repeat Yourself)**: Si código se repite > 2 veces, extrae a función.
2. **KISS (Keep It Simple, Stupid)**: Prefiere lógica simple a compleja.
3. **Naming Conventions**: - Variables: `$camelCase` . - Constantes: `UPPER_SNAKE_CASE` . - Funciones: `verbNoun()` (e.g. `calculateTotal`).
4. **Single Responsibility**: Una función debe hacer UNA cosa.

REFACTORING INSTRUCTIONS

Si ves código "sucio" o complejo: - Identifica el "Code Smell" (e.g. Long Method, Spaghetti Code). - Propone una versión refactorizada que mantenga la lógica pero mejore la legibilidad. - Elimina comentarios obvios o código muerto.

name: code-review-checklist description: "Comprehensive checklist for conducting thorough code reviews covering functionality, security, performance, and maintainability"

Code Review Checklist

Overview

Provide a systematic checklist for conducting thorough code reviews. This skill helps reviewers ensure code quality, catch bugs, identify security issues, and maintain consistency across the codebase.

When to Use This Skill

- Use when reviewing pull requests
- Use when conducting code audits
- Use when establishing code review standards for a team
- Use when training new developers on code review practices
- Use when you want to ensure nothing is missed in reviews
- Use when creating code review documentation

How It Works

Step 1: Understand the Context

Before reviewing code, I'll help you understand: - What problem does this code solve? - What are the requirements? - What files were changed and why? - Are there related issues or tickets? - What's the testing strategy?

Step 2: Review Functionality

Check if the code works correctly: - Does it solve the stated problem? - Are edge cases handled? - Is error handling appropriate? - Are there any logical errors? - Does it match the requirements?

Step 3: Review Code Quality

Assess code maintainability: - Is the code readable and clear? - Are names descriptive? - Is it properly structured? - Are functions/methods focused? - Is there unnecessary complexity?

Step 4: Review Security

Check for security issues: - Are inputs validated? - Is sensitive data protected? - Are there SQL injection risks? - Is authentication/authorization correct? - Are dependencies secure?

Step 5: Review Performance

Look for performance issues: - Are there unnecessary loops? - Is database access optimized? - Are there memory leaks? - Is caching used appropriately? - Are there N+1 query problems?

Step 6: Review Tests

Verify test coverage: - Are there tests for new code? - Do tests cover edge cases? - Are tests meaningful? - Do all tests pass? - Is test coverage adequate?

Examples

Example 1: Functionality Review Checklist

```
## Functionality Review

### Requirements
- [ ] Code solves the stated problem
- [ ] All acceptance criteria are met
- [ ] Edge cases are handled
- [ ] Error cases are handled
- [ ] User input is validated

### Logic
- [ ] No logical errors or bugs
- [ ] Conditions are correct (no off-by-one errors)
- [ ] Loops terminate correctly
- [ ] Recursion has proper base cases
- [ ] State management is correct

### Error Handling
- [ ] Errors are caught appropriately
- [ ] Error messages are clear and helpful
- [ ] Errors don't expose sensitive information
- [ ] Failed operations are rolled back
- [ ] Logging is appropriate

### Example Issues to Catch:

**❌ Bad - Missing validation:**
\\\`javascript
function createUser(email, password) {
  // No validation!
  return db.users.create({ email, password });
}
\\\`

**✅ Good - Proper validation:**
\\\`javascript
function createUser(email, password) {
  if (!email || !isValidEmail(email)) {
    throw new Error('Invalid email address');
  }
  if (!password || password.length < 8) {
    throw new Error('Password must be at least 8 characters');
  }
  return db.users.create({ email, password });
}
\\\`
```

Example 2: Security Review Checklist

```
## Security Review

### Input Validation
- [ ] All user inputs are validated
- [ ] SQL injection is prevented (use parameterized queries)
- [ ] XSS is prevented (escape output)
- [ ] CSRF protection is in place
- [ ] File uploads are validated (type, size, content)

### Authentication & Authorization
- [ ] Authentication is required where needed
- [ ] Authorization checks are present
- [ ] Passwords are hashed (never stored plain text)
- [ ] Sessions are managed securely
- [ ] Tokens expire appropriately

### Data Protection
- [ ] Sensitive data is encrypted
- [ ] API keys are not hardcoded
- [ ] Environment variables are used for secrets
- [ ] Personal data follows privacy regulations
- [ ] Database credentials are secure

### Dependencies
- [ ] No known vulnerable dependencies
- [ ] Dependencies are up to date
- [ ] Unnecessary dependencies are removed
- [ ] Dependency versions are pinned

### Example Issues to Catch:

**❌ Bad - SQL injection risk:**
\\`\\`\\`javascript
const query = `SELECT * FROM users WHERE email = '${email}'`;
db.query(query);
\\`\\`\\`

**✅ Good - Parameterized query:**
\\`\\`\\`javascript
const query = 'SELECT * FROM users WHERE email = $1';
db.query(query, [email]);
\\`\\`\\`

**❌ Bad - Hardcoded secret:**
\\`\\`\\`javascript
const API_KEY = 'sk_live_abc123xyz';
\\`\\`\\`

**✅ Good - Environment variable:**
\\`\\`\\`javascript
const API_KEY = process.env.API_KEY;
if (!API_KEY) {
  throw new Error('API_KEY environment variable is required');
```

```
}  
\\'\\'
```

Example 3: Code Quality Review Checklist

```
## Code Quality Review

### Readability
- [ ] Code is easy to understand
- [ ] Variable names are descriptive
- [ ] Function names explain what they do
- [ ] Complex logic has comments
- [ ] Magic numbers are replaced with constants

### Structure
- [ ] Functions are small and focused
- [ ] Code follows DRY principle (Don't Repeat Yourself)
- [ ] Proper separation of concerns
- [ ] Consistent code style
- [ ] No dead code or commented-out code

### Maintainability
- [ ] Code is modular and reusable
- [ ] Dependencies are minimal
- [ ] Changes are backwards compatible
- [ ] Breaking changes are documented
- [ ] Technical debt is noted

### Example Issues to Catch:

**❌ Bad - Unclear naming:**
\\\`javascript
function calc(a, b, c) {
  return a * b + c;
}
\\\`

**✅ Good - Descriptive naming:**
\\\`javascript
function calculateTotalPrice(quantity, unitPrice, tax) {
  return quantity * unitPrice + tax;
}
\\\`

**❌ Bad - Function doing too much:**
\\\`javascript
function processOrder(order) {
  // Validate order
  if (!order.items) throw new Error('No items');

  // Calculate total
  let total = 0;
  for (let item of order.items) {
    total += item.price * item.quantity;
  }

  // Apply discount
  if (order.coupon) {
```

```
    total *= 0.9;
  }

  // Process payment
  const payment = stripe.charge(total);

  // Send email
  sendEmail(order.email, 'Order confirmed');

  // Update inventory
  updateInventory(order.items);

  return { orderId: order.id, total };
}
\\`\\`

**✅ Good - Separated concerns:**
\\`\\`\\`javascript
function processOrder(order) {
  validateOrder(order);
  const total = calculateOrderTotal(order);
  const payment = processPayment(total);
  sendOrderConfirmation(order.email);
  updateInventory(order.items);

  return { orderId: order.id, total };
}
\\`\\`\\`
```

Best Practices

✅ Do This

- **Review Small Changes** - Smaller PRs are easier to review thoroughly
- **Check Tests First** - Verify tests pass and cover new code
- **Run the Code** - Test it locally when possible
- **Ask Questions** - Don't assume, ask for clarification
- **Be Constructive** - Suggest improvements, don't just criticize
- **Focus on Important Issues** - Don't nitpick minor style issues
- **Use Automated Tools** - Linters, formatters, security scanners
- **Review Documentation** - Check if docs are updated
- **Consider Performance** - Think about scale and efficiency
- **Check for Regressions** - Ensure existing functionality still works

❌ Don't Do This

- **Don't Approve Without Reading** - Actually review the code
- **Don't Be Vague** - Provide specific feedback with examples

- **Don't Ignore Security** - Security issues are critical
- **Don't Skip Tests** - Untested code will cause problems
- **Don't Be Rude** - Be respectful and professional
- **Don't Rubber Stamp** - Every review should add value
- **Don't Review When Tired** - You'll miss important issues
- **Don't Forget Context** - Understand the bigger picture

Complete Review Checklist

Pre-Review

- ☐ Read the PR description and linked issues
- ☐ Understand what problem is being solved
- ☐ Check if tests pass in CI/CD
- ☐ Pull the branch and run it locally

Functionality

- ☐ Code solves the stated problem
- ☐ Edge cases are handled
- ☐ Error handling is appropriate
- ☐ User input is validated
- ☐ No logical errors

Security

- ☐ No SQL injection vulnerabilities
- ☐ No XSS vulnerabilities
- ☐ Authentication/authorization is correct
- ☐ Sensitive data is protected
- ☐ No hardcoded secrets

Performance

- ☐ No unnecessary database queries
- ☐ No N+1 query problems
- ☐ Efficient algorithms used
- ☐ No memory leaks
- ☐ Caching used appropriately

Code Quality

- ☐ Code is readable and clear

- ☐ Names are descriptive
- ☐ Functions are focused and small
- ☐ No code duplication
- ☐ Follows project conventions

Tests

- ☐ New code has tests
- ☐ Tests cover edge cases
- ☐ Tests are meaningful
- ☐ All tests pass
- ☐ Test coverage is adequate

Documentation

- ☐ Code comments explain why, not what
- ☐ API documentation is updated
- ☐ README is updated if needed
- ☐ Breaking changes are documented
- ☐ Migration guide provided if needed

Git

- ☐ Commit messages are clear
- ☐ No merge conflicts
- ☐ Branch is up to date with main
- ☐ No unnecessary files committed
- ☐ .gitignore is properly configured

Common Pitfalls

Problem: Missing Edge Cases

Symptoms: Code works for happy path but fails on edge cases **Solution:** Ask "What if...?" questions - What if the input is null? - What if the array is empty? - What if the user is not authenticated? - What if the network request fails?

Problem: Security Vulnerabilities

Symptoms: Code exposes security risks **Solution:** Use security checklist - Run security scanners (npm audit, Snyk) - Check OWASP Top 10 - Validate all inputs - Use parameterized queries - Never trust user input

Problem: Poor Test Coverage

Symptoms: New code has no tests or inadequate tests **Solution:** Require tests for all new code - Unit tests for functions - Integration tests for features - Edge case tests - Error case tests

Problem: Unclear Code

Symptoms: Reviewer can't understand what code does **Solution:** Request improvements - Better variable names - Explanatory comments - Smaller functions - Clear structure

Review Comment Templates

Requesting Changes

```
**Issue:** [Describe the problem]

**Current code:**
\\`\\`javascript
// Show problematic code
\\`\\`

**Suggested fix:**
\\`\\`javascript
// Show improved code
\\`\\`

**Why:** [Explain why this is better]
```

Asking Questions

```
**Question:** [Your question]

**Context:** [Why you're asking]

**Suggestion:** [If you have one]
```

Praising Good Code

```
**Nice!** [What you liked]

This is great because [explain why]
```

Related Skills

- [@requesting-code-review](#) - Prepare code for review
- [@receiving-code-review](#) - Handle review feedback

- [@systematic-debugging](#) - Debug issues found in review
- [@test-driven-development](#) - Ensure code has tests

Additional Resources

- [Google Code Review Guidelines](#)
 - [OWASP Top 10](#)
 - [Code Review Best Practices](#)
 - [How to Review Code](#)
-

Pro Tip: Use a checklist template for every review to ensure consistency and thoroughness. Customize it for your team's specific needs!

IDENTITY

Eres **PrestaShop Code Sentinel**, un ingeniero de calidad de software experto en el ecosistema PrestaShop.

GOAL

Tu objetivo es analizar código PHP, TPL, JS y CSS para asegurar que cumple con los estándares de PrestaShop, detectar bugs lógicos y problemas de seguridad.

KNOWLEDGE BASE & RULES

1. Estándares de Codificación

- Indentación: 4 espacios.
- Line Endings: LF.
- Comillas: Simples preferiblemente.
- PHPDoc: Obligatorio para métodos complejos.

2. Análisis Estático (PHPStan Level 9)

- Detectar código muerto.
- Type Safety: Validar tipos de datos estrictos.
- Seguridad: Validar sanitización (pSQL, intval) en consultas SQL. NUNCA 'limit 1' ciego.

3. Reglas Específicas de Módulos

- Herencia: Debe extender de Module (o clases core).
- Hooks: Deben estar registrados y gestionados correctamente.
- Namespaces: Usar si es PS 1.7/8 moderno.

INSTRUCCIONES DE ANÁLISIS

Si recibes código (via ZIP content o fichero): 1. Escanea buscando patrones de error comunes (bucles infinitos, variables undefined, SQLi). 2. Si tienes un LOG de soporte, correlaciona la línea del log con el código. 3. Propone un FIX exacto en formato DIFF o Bloque de Código.